

Being open source (and the only independent open source Object Relational DataBase Management System—ORDBMS), users of PostgreSQL are not limited by its functionality. They are free to contribute the required features to the PostgreSQL community or maintain their own version of PostgreSQL code with modifications. Owing to the extensibility of PostgreSQL, this is not always needed. In this article, we will discuss various ways in which the functionality of PostgreSQL can be extended by using SQL APIs, without changing the core server code.

User-defined functions

PostgreSQL allows users to define their own functions using the `CREATE FUNCTION` command. These functions can be written in SQL, or in built-in or user-defined procedural languages. They can accept arguments of built-in data-types or user-defined data-types and return nothing, or objects of built-in data-types and user-defined data-types. User-defined functions may read or write data from the database, and can be overloaded. User-defined functions form an essential ingredient in the rest of the extensibility features. Although user-defined functions are common to most database systems, PostgreSQL goes a step further and allows functions written in C and stored in shared libraries to be called from the SQL interface.

User-defined type systems

PostgreSQL allows users to define their own data-types, which can be used in the same way as the in-built data-types. A user-defined data-type can be a base type, a *composite* type, a domain or an enumeration. User-defined types can be defined by using the `CREATE TYPE` (base, composite and enumeration) or `CREATE DOMAIN` (domain) commands.

Composite types are a collection of other data-types with associated field names, like structures in C. The members of a composite type can be accessed in SQL queries, functions, etc. An example defining a composite type 'complex' can be found in this article.

Domain types, on the other hand, can be created by constraining the base types. Although the constraints restrict the valid values of a domain type to a subset of values allowed by the underlying base type, for many purposes, a base type and the types based on it are interchangeable.

Base types are usually implemented in a low-level language like C, which is below the level of the SQL language. The behaviour of such a data-type needs to be defined by the user in terms of input/output functions (usually inverses of each other), which decide how the data-type appears in string form for input/output by the user. The same functions will decide how the type is organised in memory. The implementation can be provided through shared libraries.

While creating a base type, users can provide specifications for storage, statistical analysis, collation, etc, for better performance and usage.

User-defined types can be extended further by defining operators and casts operating on these types, using the `CREATE OPERATOR` and `CREATE CAST` commands. Users can also define new operators for existing data-types. An operator is syntactic sugar for a call to the actual function that performs the 'operation'. Thus, a user needs to define the underlying function before the operator can be defined. Operators can be overloaded, i.e., the same operator name (a string containing characters like '+', '-', '>', '=', etc, following certain rules) can be used for different operations depending upon the types and number of operands. An operator specification goes beyond mere 'syntactic sugar', giving additional information that helps query planner optimise queries, involving that operator. Users can specify: a commutative operator, allowing query optimisers to 'flip around' sides of the operator; or negator operator, to 'absorb' NOT operator; or restriction selectivity estimator, to estimate the number of rows that satisfy conditions involving the operator; or whether join conditions involving the operator can be hash or merge strategies etc.

Going a step further, it is possible to define operator classes to build the B-tree indices on the columns with user-defined data-types. This set of features allows you to seamlessly use user-defined data-types in the same way as PostgreSQL built-in types.

User-defined aggregates

Aggregates are an important part of the queries used for report generation and analytics. PostgreSQL allows users to define aggregates accepting arguments of built-in as well as user-defined data-types. The aggregation process in PostgreSQL takes place in two steps. First, a transition result is calculated by processing every row participating in the aggregate, whereas the second step processes the transition result to get the final aggregation result. Some aggregate functions like `SUM` do not need the second step; hence, it is optional. A user-defined aggregate specification essentially contains the functions to be used in the aggregation steps, and the data-types of the inputs to, and the output from, these functions. Aggregates can be defined using the `CREATE AGGREGATE` command.

Example: Putting it all together

This section describes a simple implementation of a data-type 'complex' in PostgreSQL. In this example, we will create a data-type complex as a composite type, and will add some operators and aggregation functions to it. A better implementation can be found in the standard distribution of PostgreSQL source code, under the `src/tutorial` directory.